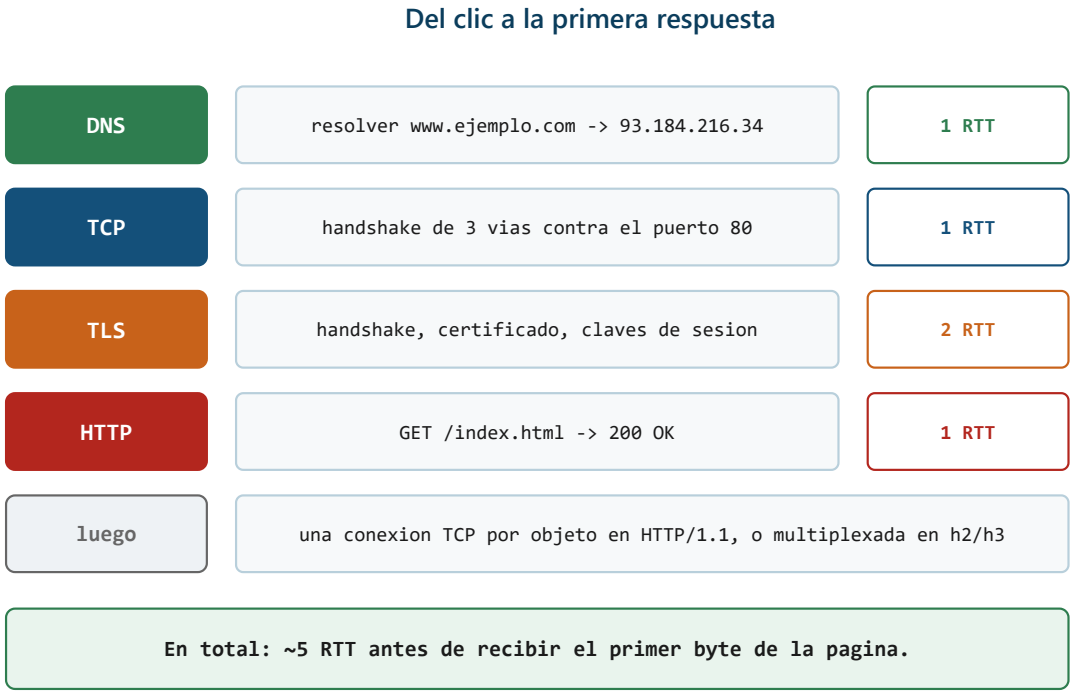
Clase 9 — HTTP a fondo: mensajes, cookies, caché y versiones

Bloque: B (Aplicación) · **Semana:** 3 · **Duración estimada:** 110 min **Correlativa:** Clase 8 (modelo cliente/servidor y HTTP/1.1) · **Sigue con:** Clase 10 (DNS: la base de datos distribuida) **Material de referencia:** Tanenbaum cap. 7 (§7.3) · RFC 9110, RFC 9112, RFC 9113, RFC 9114, RFC 6265, RFC 7541, RFC 9204, RFC 9218, RFC 7838, RFC 9000



🎯 Al terminar esta clase vas a poder

1. Enumerar los **métodos** de HTTP y explicar qué significa que uno sea **seguro** o **idempotente**, y por qué esa distinción decide si un cliente puede reintentar solo.
2. Escribir de memoria un **intercambio HTTP/1.1 completo** de validación condicional que termina en **304**, con `ETag`, `If-None-Match` y todas las cabeceras necesarias.
3. Explicar qué hace **cada** cabecera de la lista que hay que saber, sin consultarla.
4. Diferenciar `no-store`, `no-cache`, `max-age`, `private` y `public`, y explicar qué pasa exactamente con `max-age=0` cuando hay un `ETag` presente.
5. Distinguir un **proxy**, un **gateway** y un **CDN**, y explicar por qué un CDN mejora la latencia con el concepto de **distancia al usuario**.
6. Explicar qué **agrega HTTPS**, y por qué eso no es lo mismo que cambiar de versión de HTTP.
7. Comparar **HTTP/1.1, HTTP/2 y HTTP/3** diciendo qué problema exacto resuelve cada una, qué parte del *head-of-line blocking* queda y qué parte desaparece, con las cifras reales.

8. Enumerar **qué sigue igual** en las tres versiones, porque es lo que el examen da por sabido.

> **Prerequisito del examen:** `2010-07 P1` (8 puntos — la carga de una página, repreguntado acá con el ángulo de versiones), `2009-07 Problema 2` (30 puntos — el cliente anti-spam que habla HTTP), y todo lo que la cátedra pregunta sobre «qué versión de HTTP es la actual».

PARTE 1 — TEORÍA

1.1 Los métodos, uno por uno

Un método HTTP no es un verbo decorativo: **es una declaración de intención sobre el recurso**. El servidor la usa para decidir si puede cachear, si tiene que autenticar y qué hacer con el cuerpo.

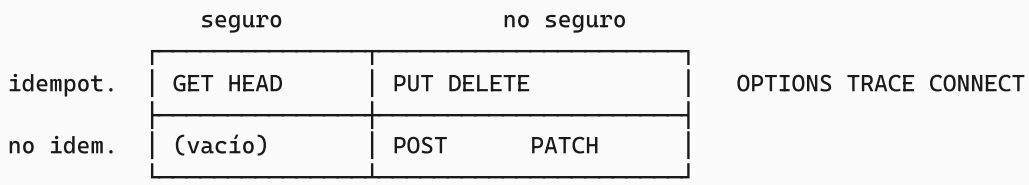
Método	Qué hace	Cuerpo	Típica	Notas
GET	**Obtiene** un recurso	prohibido (se tolera)	`200`	Seguro e idempotente. El único cacheable por definición.
HEAD	Igual que GET **sin cuerpo**	no	`200` sin cuerpo	Seguro e idempotente. Permite conocer el `Content-Length` sin bajarlo.
POST	**Crea** o dispara una acción	**obligatorio**	`201` o `200`	**Ni seguro ni idempotente.** El único donde el cuerpo define la operación.
PUT	**Reemplaza** el recurso	obligatorio	`200` / `204` / `201`	Idempotente, **no seguro**. La URL identifica, el cuerpo define.
DELETE	**Borra** el recurso	opcional	`204`	Idempotente, no seguro. Borrar dos veces es lo mismo que borrar una.
OPTIONS	Pregunta **qué admite**	no	`200` + `Allow:`	Seguro e idempotente. Descubre capacidades (CORS).
TRACE	Devuelve el mensaje **tal como llegó**	no	`200` con el eco	Seguro e idempotente. Diagnóstico de proxies. Casi nadie lo implementa.
CONNECT	Abre un **túnel TCP**	no	`200`	Seguro e idempotente. Base de los proxies y de HTTPS.
PATCH	**Modificación parcial**	obligatorio	`200` / `204` / `202`	Ni seguro ni idempotente. **No existía en HTTP/1.0**.

Tres detalles de memoria:

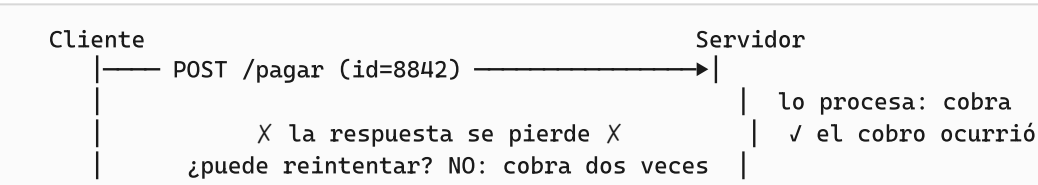
1. `GET` **con cuerpo.** La especificación permite que el servidor lo rechace (`400` o `413`) y muchos lo hacen. La razón histórica: los caches y proxies de la época solo cacheaban peticiones **sin** cuerpo, porque sin cuerpo la identidad de la petición se reduce a `(método, URL)`. Con cuerpo, dos peticiones idénticas pueden tener semántica distinta.
2. `PUT` **reemplaza,** `POST` **crea.** En `PUT` la URL **ya identifica** el recurso y el cuerpo dice qué tiene que contener. En `POST` la URL identifica **una colección** y el servidor **decide** la URL del recurso nuevo: por eso devuelve `201 Created` con `Location`.
3. `CONNECT` **no es una petición HTTP normal.** Su respuesta es una conexión TCP **cruda** que queda abierta: lo que venga después no es HTTP (típicamente un `ClientHello` de TLS).

1.2 Seguros e idempotentes: por qué la distinción importa

Propiedad	Definición	Métodos que la tienen
Seguro (*safe*)	**No modifica el estado del servidor.**	`GET`, `HEAD`, `OPTIONS`, `TRACE`, `CONNECT`
Idempotente	Repetir la misma petición *n* veces produce **el mismo efecto** que repetirla una. No significa la misma respuesta byte a byte: significa el **mismo efecto sobre el estado**.	`GET`, `HEAD`, `PUT`, `DELETE`, `OPTIONS`, `TRACE`, `CONNECT`



El problema que resuelve es el **reintento por timeout**. TCP garantiza la entrega, pero HTTP no sabe si la **respuesta** llegó, y hay caminos en los que tampoco puede saber si la **petición** fue procesada:



La regla de los clientes HTTP reales: si el método es **idempotente**, se puede reintentar automáticamente; si es `POST`, no se reintenta solo: o se le pregunta al usuario, o se manda una **idempotency key** que el servidor usa para deduplicar.

El razonamiento para el examen: las dos propiedades son del **efecto**, no de la intención. `DELETE` **no es seguro** (borra algo) pero **sí es idempotente** (borrar un recurso ya borrado no cambia nada más). `POST` es las dos cosas negativas **a la vez**, y por eso es el único con cuerpo obligatorio.

Matiz que suma puntos: un `GET` **sí** puede modificar el servidor (contador de visitas, *log*, token rotado). Los métodos seguros **deben** ser de solo lectura, pero el resumen de uso puede registrar. Lo que la propiedad garantiza es que **no es la intención del protocolo** cambiar el recurso.

1.3 POST vs. GET: dónde van los datos

```
GET /buscar?q=gorila&lang=es HTTP/1.1          ⟵ los datos van EN LA URL
Host: www.fing.edu.uy

POST /buscar HTTP/1.1                          ⟵ los datos van EN EL CUERPO
Host: www.fing.edu.uy
Content-Type: application/x-www-form-urlencoded
Content-Length: 21

q=gorila&lang=es
```

	`GET`	`POST`
Dónde van los datos	en la **URL**, tras el `?`, separados por `&`	en el **cuerpo**, tras la línea en blanco
Qué los delimita	el `?`, el `&` y el `=`	el `Content-Type` (cómo se codifican) y el `Content-Length`
Tamaño	el de la URL: ~2000 a 8000 caracteres	el del cuerpo; el límite lo pone `413 Payload Too Large`
Visibilidad	queda en el historial, en los logs del servidor, en el `Referer`	**no** aparece en la URL ni en el historial
Cacheable	**sí**, por definición	**no**, salvo `Cache-Control` explícito
Idempotente	sí	**no**
Se repite con un link	sí	no: un formulario `POST` no se reenvía al pulsar «atrás»

El argumento de seguridad: mandar una contraseña en la URL es un error grave, porque queda escrita en el **historial del navegador**, en los **logs de acceso** del servidor, en el `Referer` que se le manda a los terceros y en todos los **proxies** del camino. Todo eso en texto plano en disco. Por eso los formularios de login usan `POST`.

La razón técnica: la URL es la **identidad** del recurso en HTTP. `GET /articulos/2026/10` y `GET /articulos/2026/11` son dos recursos distintos y por eso dos entradas distintas de la caché. El `POST` no identifica un recurso: **invoca un proceso**. Esa es la razón de fondo de que no sea ni seguro ni idempotente.

1.4 Las cabeceras que hay que saber

Cabeceras de la **petición**:

Cabecera	Qué hace	Ejemplo	Por qué importa
`Host`	**Obligatoria en HTTP/1.1.** Dice a qué servidor del destino se pide.	`Host: www.fing.edu.uy`	Hace posible el *virtual hosting*: un solo `132.42.120.5` aloja `fing.edu.uy`, `ing.edu.uy` y `redescomp.uy`. **Es la razón de que HTTP/1.1 no pueda esconderse detrás de un NAT.**
`User-Agent`	Identifica el cliente.	`Mozilla/5.0 (X11; Linux x86_64)`	Muchos servidores devuelven contenido distinto según esto, y los bots se identifican y se los bloquea.
`Accept`	Qué **tipos de contenido** acepta.	`Accept: text/html,application/xhtml+xml`	Permite la **negociación de contenido**.
`Accept-Encoding`	Qué **compresiones** acepta.	`Accept-Encoding: gzip, deflate, br`	Si acepta, el servidor responde `Content-Encoding: gzip`.
`Accept-Language`	Qué **idiomas** prefiere.	`Accept-Language: es-UY,es;q=0.9`	El servidor devuelve la versión en español.
`Cookie`	Devuelve las cookies del cliente.	`Cookie: sesion=abc123`	El estado viaja **de vuelta**.
`Authorization`	Credenciales.	`Authorization: Bearer eyJhbGci...`	**Nunca en texto plano sobre HTTP.**
`Connection`	Control de la persistencia.	`Connection: close`	En HTTP/1.1 el default es persistente; el `close` es la excepción.
`If-None-Match`	Condición: «solo si el `ETag` cambió».	`If-None-Match: "3f2a1c8"`	Es lo que dispara el **304**.
`If-Modified-Since`	Condición por **fecha**.	`If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT`	Alternativa más gruesa al `ETag`.
`If-Range`	«Si no cambió, haceme un `Range`».	`If-Range: "3f2a1c8"`	Permite **reanudar** una descarga sin corromper el archivo.
`Range`	Pide **un fragmento**.	`Range: bytes=2048-4095`	Base del video streaming y de las descargas reanudables.
`Origin`	De dónde viene la petición *cross-origin*.	`Origin: https://otro.edu.uy`	Con `Access-Control-Allow-Origin` implementa CORS.

Cabeceras de la **respuesta**:

Cabecera	Qué hace	Ejemplo
`Date`	Fecha y hora del servidor, **RFC 1123**, siempre en **GMT/UTC**.	`Date: Tue, 06 Oct 2026 14:22:31 GMT`
`Server`	Qué software atiende.	`Server: Apache/2.4.62 (Ubuntu)` — información de ataque gratuita.
`Content-Type`	**Qué es** el cuerpo.	`text/html; charset=utf-8`, `image/png`
`Content-Length`	**Cuántos bytes** tiene el cuerpo.	`Content-Length: 4096` — permite saber dónde termina la respuesta sin cerrar.
`Content-Encoding`	Qué compresión se aplicó.	`Content-Encoding: gzip`
`Transfer-Encoding`	Cómo se codifica **la transferencia**.	`Transfer-Encoding: chunked` — ver abajo.
`Location`	Dónde está o quedó el recurso.	`Location: /articulos/2026/10/creado` — obligatorio en un `201`.
`Allow`	Métodos y opciones admitidos.	`Allow: GET, HEAD, POST, OPTIONS`
`ETag`	**Etiqueta de versión** del recurso.	`ETag: "3f2a1c8-9b41-6512d3f0"`
`Last-Modified`	Última modificación.	`Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT`
`Cache-Control`	**Instrucciones de caché**.	`Cache-Control: max-age=600, private`
`Expires`	Fecha de vencimiento (mecanismo viejo).	`Expires: Tue, 06 Oct 2026 14:32:31 GMT` — hoy se prefiere `Cache-Control`.
`Vary`	**De qué depende** la variación de la respuesta.	`Vary: Accept-Encoding`
`Age`	Cuántos segundos lleva en la **caché de un intermediario**.	`Age: 124`
`Set-Cookie`	El servidor **crea** una cookie.	`Set-Cookie: sesion=abc123; Path=/; HttpOnly; Secure`
`WWW-Authenticate`	Qué esquema usar tras un `401`.	`WWW-Authenticate: Bearer realm="fing"`
`Access-Control-Allow-Origin`	A quién se le permite leer la respuesta.	`Access-Control-Allow-Origin: https://fing.edu.uy`

`Transfer-Encoding: chunked`

Si el servidor **no sabe de antemano** el tamaño del cuerpo (porque lo genera en streaming), no puede mandar `Content-Length`. La salida es mandar el cuerpo **fragmentado**:

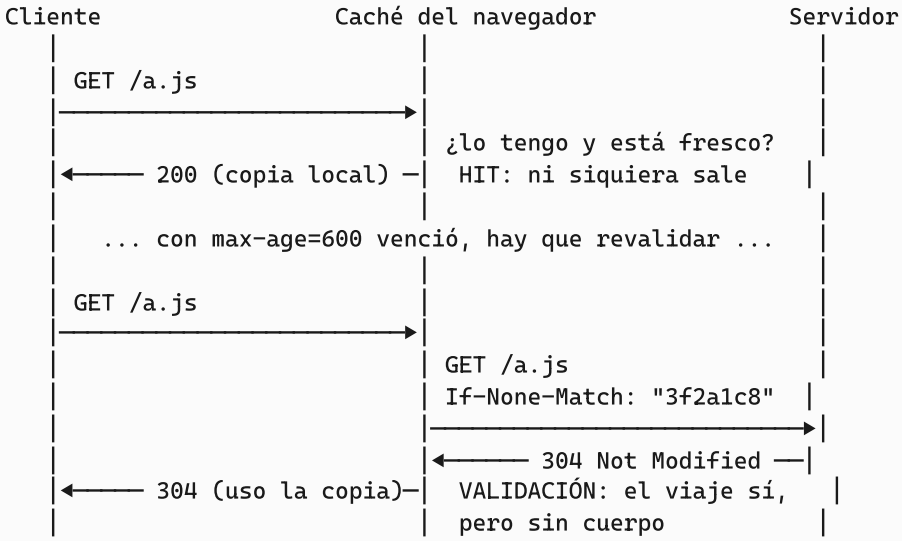
```
HTTP/1.1 200 OK
Content-Type: text/plain
Transfer-Encoding: chunked

1a
Hola Mun
14
do: soy un cuerpo
0
```

Cada fragmento es `<tamaño en hexadecimal><CRLF>`, `<datos><CRLF>`. El `0` en hexadecimal es el **fin**. **Regla dura:** si mandás `Transfer-Encoding`, **no podés** mandar `Content-Length` al mismo tiempo, porque son dos formas distintas de decir dónde termina el cuerpo y el cliente no sabe cuál obedecer.

1.5 La caché HTTP

La caché existe porque **cargar una página es, mayoritariamente, esperar**. Ataca las dos fuentes de espera: **evita el viaje de ida y vuelta** cuando hay un *hit* fresco, y **si tiene que ir al servidor, lo hace con una validación condicional** (un `304` pesa 180 bytes en vez de 4 MB).



Directiva	Significado exacto	Consecuencia práctica
`no-store`	**No lo guardes en ninguna caché**: ni memoria, ni disco, ni intermediario.	Es lo que se pone en un banco. **Es la única que garantiza confidencialidad del dato en reposo.**
`no-cache`	**Guárdalo, pero no lo sirvas sin revalidarlo.**	Contradicción aparente e intencional: se cachea para no perder la copia, pero nunca se usa sin preguntar.
`max-age=N`	Fresco durante **N segundos**.	El TTL explícito de HTTP; desplaza al `Expires`.
`s-maxage=N`	Igual, pero **solo para cachés compartidas** (proxies, CDN).	Permite que un proxy tenga una ventana distinta de la del navegador.
`private`	**Solo** la caché del navegador puede guardar esto; los proxies compartidos **no**.	Obligatorio en toda respuesta con datos de usuario: si no, un proxy corporativo podría servirle el correo de otro usuario.
`public`	Se puede guardar en cachés **compartidas**.	Para contenido igual para todos, con caching agresivo.
`no-transform`	Ningún proxy puede cambiar el cuerpo (ni comprimirlo).	Para contenido ya comprimido o firmado.
`must-revalidate`	Vencido, **no** se puede servir ni del *stale*.	Se combina con `proxy-revalidate`.
`stale-while-revalidate=N`	Si está vencido, **servilo igual** y revalidá en segundo plano.	Reduce la latencia percibida en sitios con mucho tráfico.
`stale-if-error=N`	Si el origen falla, serví el viejo hasta `N` segundos.	Resiliencia.

El contraste que hay que entender:

```
no-store    → "no lo guardes"           (ni memoria ni disco)
no-cache    → "guardalo, pero revalidá" (memoria sí, disco sí)
max-age=0   → "fresco por 0 segundos"   ≡ revalidar SIEMPRE
```

`no-cache` y `max-age=0` terminan **haciendo lo mismo** en cuanto al uso, pero se diferencian en la **retención**: con `no-cache` el recurso queda y sirve si el servidor se cae; con `max-age=0` también queda, pero está vencido desde el minuto uno.

`Vary` y `Age`, los dos que casi nadie se acuerda. `Vary` **no dice cuánto tiempo vale**: dice **bajo qué claves** la respuesta es distinta.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Encoding: gzip
Vary: Accept-Encoding
Cache-Control: max-age=600

┌──────────────────────────────────────────────────────┐
│  LA CACHÉ GUARDA DOS ENTRADAS:                       │
│    ( /index.html , Accept-Encoding: gzip ) → 200, 4096 B  │
│    ( /index.html , Accept-Encoding: br )   → 200, 3802 B  │
└──────────────────────────────────────────────────────┘
```

Sin `Vary`, la caché guarda **una sola** entrada y le devuelve al cliente que no soporta gzip **el cuerpo comprimido**: la página no abre. Es un bug real.

`Age` completa el modelo: cuando una respuesta pasa por un proxy, el proxy **resta el tiempo que estuvo cacheada**:

```
frescura restante = max-age − Age

Servidor manda:                    Cache-Control: max-age=600
El proxy lo tuvo 250 s y reenvía:  Age: 250
→ al cliente final le quedan 600 − 250 = 350 s de frescura
```

1.6 `ETag` + `Last-Modified`: la validación condicional

	`Last-Modified`	`ETag`
Es	una **fecha**	una **cadena opaca** que el servidor inventa
Precisión	**1 segundo**	la que el servidor quiera
Problema	dos versiones en el mismo segundo son **indistinguibles**	ninguno: es opaco y único por diseño
Ejemplo	`Mon, 05 Oct 2026 09:14:07 GMT`	`"3f2a1c8-9b41-6512d3f0"`
Variante débil	—	`W/"3f2a1c8"`: «no cambió semánticamente, aunque los bytes sí»

Las dos parejas que existen y hay que nombrar: `ETag` / `If-None-Match` y `Last-Modified` / `If-Modified-Since`. Si hay ambos validadores, **el `ETag` manda**.

El intercambio completo, mensaje por mensaje

Situación. El 6 de octubre a las 14:22:31 el cliente pide `/notas.pdf` y no tiene nada en caché. Una hora después vuelve a pedirlo.

Mensaje 1 — petición inicial:

```
GET /notas.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

Mensaje 2 — respuesta completa:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 14:22:31 GMT
Server: Apache/2.4.62 (Ubuntu)
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=3600
Content-Type: application/pdf
Content-Length: 1048576
Accept-Ranges: bytes
Connection: keep-alive

%PDF-1.7
... 1.048.576 bytes en total ...
%%EOF
```

El cliente guarda el cuerpo, el `ETag`, el `Last-Modified` y el `Cache-Control`, y anota la hora de entrada: 14:22:31.

Una hora después (15:22:31) el `max-age=3600` venció, así que hay que revalidar.

Mensaje 3 — petición condicional:

```
GET /notas.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
If-None-Match: "3f2a1c8-9b41-6512d3f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
Connection: keep-alive
```

`If-None-Match` dice: «si el recurso tiene esta etiqueta, no me lo mandes entero, decime solamente si sigue siendo la misma». `If-Modified-Since` es la misma pregunta por fecha. Si el cliente mandó ambos, el servidor usa el `If-None-Match`, porque distingue dos versiones del mismo segundo.

Mensaje 4 — el recurso no cambió: respuesta 304:

```
HTTP/1.1 304 Not Modified
Date: Tue, 06 Oct 2026 15:22:32 GMT
ETag: "3f2a1c8-9b41-6512d3f0"
Cache-Control: max-age=3600
```

Fijate qué NO está: no hay cuerpo (cero bytes de PDF), no hay `Content-Length` (no hay nada que medir), no hay `Content-Type` (no hay nada que tipar), no hay `Server`.

Y qué SÍ está: el `ETag`, porque el `304` **renueva la frescura**: el reloj de la caché vuelve a cero y ahora vence a las 16:22:32, no a las 15:22:32. Esa es una de las sutilezas que más se pregunta.

Y si el archivo hubiera cambiado, el mismo mensaje 3 devolvería un `200` **completo**, con `Content-Length` y `ETag` distintos.

El cálculo del ahorro

```
Con 200 (re-descarga completa):
  cabeceras ≈       250 B
  cuerpo    = 1048576 B
  ──────────────────────
  total     = 1048826 B

Con 304 (validación):
  cabeceras ≈       180 B
  cuerpo    =         0 B
  ──────────────────────
  total     =      180 B

Ahorro = 1048826 − 180 = 1048646 B
Porcentaje = 1048646 / 1048826 ≈ 99,98 %
```

Interpretación física: el `304` no es una optimización de bytes, es un **cambio de orden de magnitud**. Y la letra chica: **igual se pagó el RTT de ida y vuelta**. El `304` ahorra *ancho de banda*, no *tiempo de espera*. Para matar el tiempo de espera están HTTP/2, HTTP/3 y el CDN.

1.7 Proxies, gateways y CDNs

	Proxy (*forward proxy*)	**Gateway**	**CDN**
Qué es	un intermediario que actúa **en nombre de** un cliente	un intermediario que **traduce entre protocolos**	una red de servidores que **guardan copia del contenido**
Ejemplo	el proxy del laboratorio o de la facultad	un servidor de correo que traduce SMTP a POP3	Cloudflare, Akamai, Fastly
¿Cambia el protocolo?	no	**sí**	no
¿Guarda contenido?	a veces	no	**sí, siempre**
Para qué sirve	filtrar, auditar, autenticar, compartir una salida	traducir	**latencia y descarga del origen**

> **Ojo con «gateway» en HTTP.** En la práctica el término se usa como «puerta de entrada»: un `502 Bad Gateway` casi nunca es «el servidor traductor falló», es **«el proxy inverso no pudo hablar con el backend»**. `504 Gateway Timeout` es «el proxy esperó demasiado al backend».

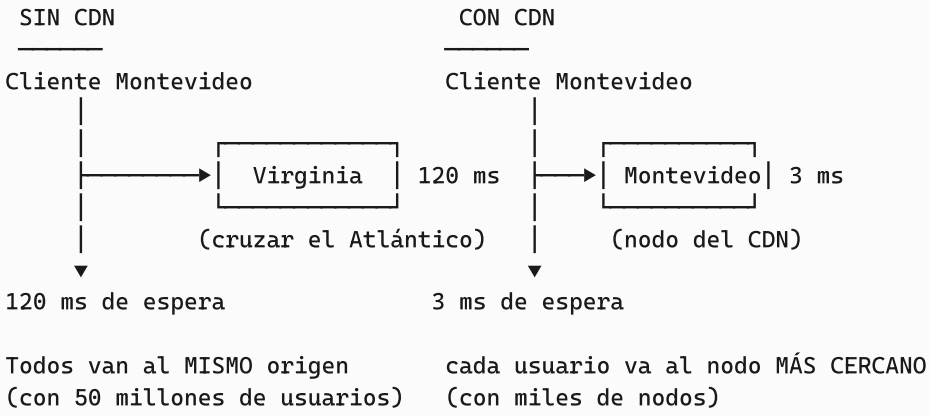
Por qué un CDN mejora la latencia: la distancia al usuario

La Clase 8 mostró que **más del 80 % del tiempo de carga de una página es espera**. Toda esa espera es tiempo de ida y vuelta:

```
t_espera ≈ (número de viajes) × RTT
y el RTT es proporcional a la distancia física y a los saltos
```

Cliente	Servidor en Virginia	Servidor en Montevideo
Montevideo	**120 ms**	**3 ms** (diferencia de 40×)
Madrid	95 ms	160 ms
Buenos Aires	130 ms	175 ms

Un **CDN** copia el contenido a un nodo **cerca** de cada usuario y responde desde ahí:



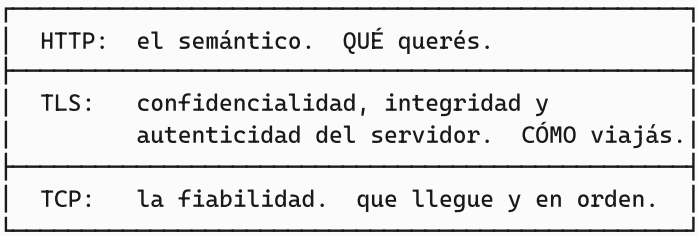
El argumento en una frase, que es lo que hay que escribir: un CDN mejora la latencia **reduciendo la distancia al usuario**, y como el tiempo de carga de una página es esencialmente `número de viajes × RTT`, bajar el RTT de 120 ms a 3 ms hace cargar la página **antes**, sin tocar un byte de contenido.

El segundo beneficio es la **descarga del origen**: si el 95 % de los pedidos lo sirve el nodo del CDN, el origen ve el 5 % y sobrevive. Por eso un CDN es también un **escudo contra DDoS**.

CDN y caché, la diferencia: la caché HTTP guarda respuestas en un punto **individual** (el navegador, un proxy). El CDN es un **sistema distribuido** de cachés coordinados entre sí: cuando un nodo recibe un pedido de algo que no tiene, **se lo pide a un par**, lo guarda y lo sirve. Eso se llama *fill on miss*, y explica por qué un CDN es barato: no duplica todo el mundo, solo lo que se pide.

1.8 HTTPS: qué agrega (y qué no es)

HTTPS es HTTP transportado por TLS. No cambia **una línea** de la semántica de HTTP: los mismos métodos, los mismos códigos, las mismas cabeceras. Lo que cambia es el **transporte**.



Propiedad	Qué aporta	Sin HTTPS	Con HTTPS
Confidencialidad	nadie en el camino ve el contenido	texto plano en cada router, ISP y proxy	**cifrado** de extremo a extremo
Integridad	nadie puede alterar el mensaje en vuelo	un intermediario puede cambiar el saldo de tu cuenta en el HTML	**AEAD**: alterar algo rompe el descifrado
Autenticación del servidor	sabés con quién estás hablando	**cero**: cualquiera que responda el puerto 80 es «el sitio»	**certificado X.509** contra una cadena de confianza
Identificación del cliente	opcional	imposible	opcional: **mTLS**

HTTPS no es una versión de HTTP. `HTTPS/3` no es «HTTP/3 con candado»: es **HTTP/3 sobre QUIC sobre TLS 1.3**, todo integrado. Y **no protege al servidor de origen** de un `502`, ni evita un *clickjacking*, ni oculta qué sitio visitaste (para eso está el DNS cifrado, Clase 11).

> ⚠️ **Adelantado:** TLS entero — cadena X.509, handshake, la separación del *cipher suite* en tres, el `0-RTT` y sus riesgos de *replay* — se ve **entero en la Clase 47**. Acá solo hace falta la capa y la frase «**HTTP aporta semántica, TLS aporta seguridad**».

1.9 De HTTP/1.1 a HTTP/2 y HTTP/3

Los tres problemas estructurales de HTTP/1.1

```
═══ (a) HOL blocking A NIVEL DE CONEXIÓN ═══

conexión 1:  GET ──► 200 ──► GET ──► 200 ──► ...
conexión 2:  GET ──► 200 ──► GET ──► 200 ──► ...
conexión 3:  GET ── X SE PIERDE ──────────────► 404, muy tarde

En HTTP/1.1 los objetos van en conexiones distintas, pero cada conexión
TCP es ORDENADA: el hueco de la conexión 3 la frena a ella entera.
ESTO ES HEAD-OF-LINE BLOCKING.


═══ (b) HOL blocking ENTRE STREAMS ═══

HTTP/2  ┬─ stream 1 ──── X SE PIERDE ──────► 404
        ├─ stream 2 ── 200 ──►
        ├─ stream 3 ─── 200 ──►

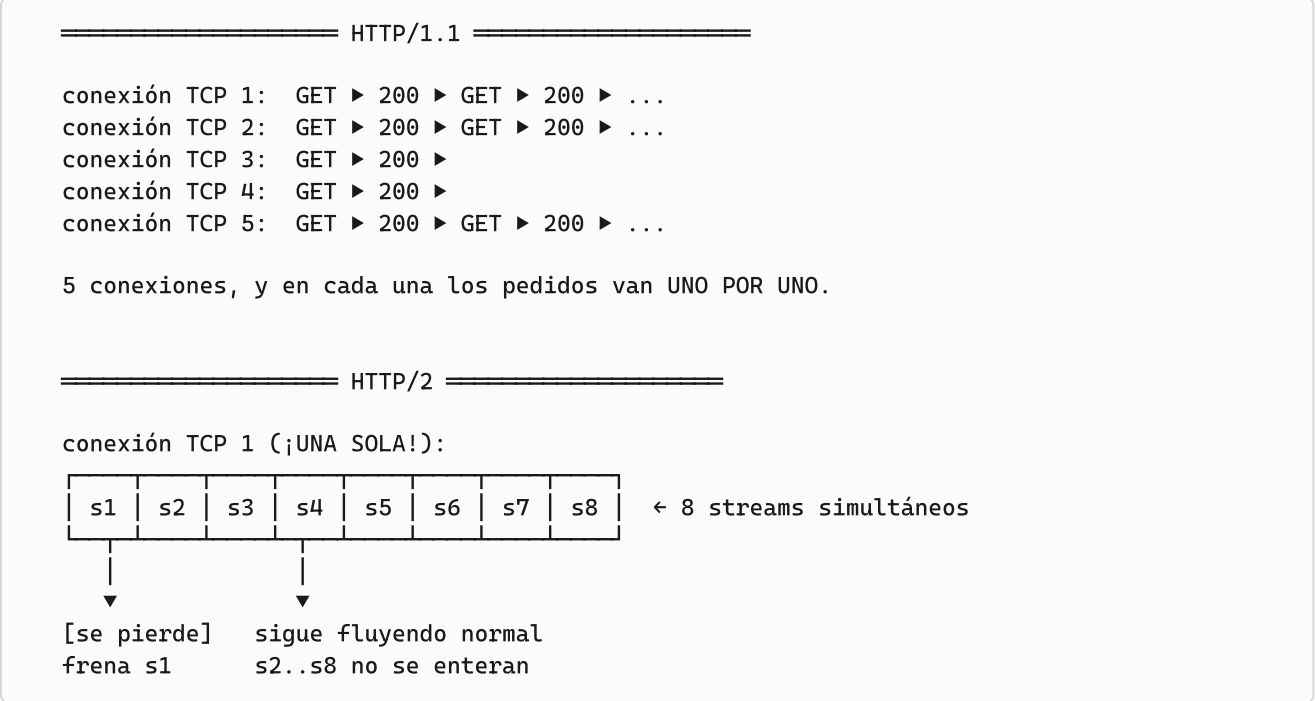
Multiplexamos: la pérdida frena SOLO al stream 1.
...pero DENTRO de cada stream sigue habiendo bloqueo, porque la conexión
TCP de abajo sigue siendo ordenada.


═══ (c) El costo de las CABECERAS ═══

Un navegador manda unas 10 cabeceras por GET y el servidor unas 8.
En HTTP/1.1 viajan en texto plano, sin comprimir, en CADA petición.
HTTP/1.1 no tiene compresión de cabeceras. HTTP/2 y HTTP/3 sí.
```

HTTP/2 (`RFC 9113` , 2022)

La idea central: **una sola conexión TCP, muchos streams independientes**.

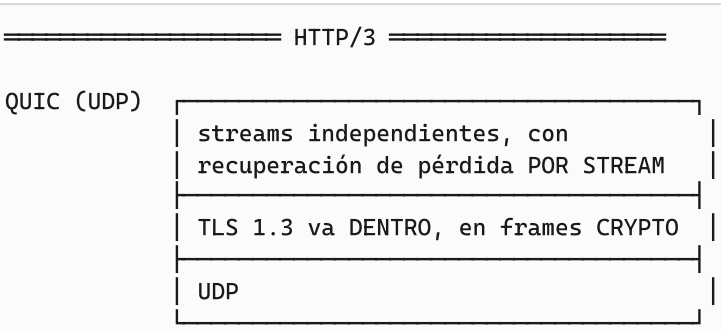


	HTTP/1.1	HTTP/2
Streams por conexión	**1** (los demás esperan)	**muchos**, simultáneos
HOL blocking a nivel de conexión	**sí**	**eliminado**
HOL blocking entre streams	no aplica	**sigue existiendo**: la pérdida de un stream frena a ese stream
Compresión de cabeceras	no	**HPACK** (`RFC 7541` , vigente)
Cabeceras en mayúsculas y repetidas	sí	se normalizan a **minúsculas** y se indexan con HPACK
Orden de los bytes	obligatorio	el **stream ID** identifica el flujo; los frames se entrelazan
Servidor que manda primero	no	**sí**, con `PUSH_PROMISE`
Prioridad	cabecera `Priority:`	la de `RFC 7540` **quedó deprecada** (§5.3 de la 9113); la reemplaza **`RFC 9218`**

El detalle que suma puntos. `RFC 7540` (HTTP/2, 2015) especificaba un esquema de prioridad con *weight* y dependencia entre streams. Al reescribir HTTP/2 en `RFC 9113` , ese esquema **se deprecó**: la razón es que **en la práctica casi nadie lo usaba** y las implementaciones divergían. El frame `PRIORITY` sobrevive solo por compatibilidad. Lo que lo reemplaza es **`RFC 9218`** **— *Extensible Prioritization Scheme* (EPS)**, que separa la **importancia semántica** del stream (`urgency`) de la **dependencia** entre unos y otros, con sintaxis textual en vez de números de peso opacos. Lección de protocolo: **especificar una característica de prioridad, descubrir que nadie la implementa y deprecarla, es un resultado normal.**

HTTP/3 (`RFC 9114`) sobre QUIC (`RFC 9000`)

HTTP/3 **no es «HTTP/2 sobre UDP»**. Reemplaza **la conexión TCP completa** por **QUIC**, un transporte nuevo con su propia recuperación de pérdida, su propio control de congestión y su propio PLPMTUD.




```
NO hay handshake de 3 vías.
NO hay conexión TCP.
NO hay retransmisión "de la conexión": cada stream se recupera solo.
NO hay identidad por IP: el Connection ID permite cambiar de IP y de
puerto sin cortar la sesión (esto arregla el NAT rebinding).
```

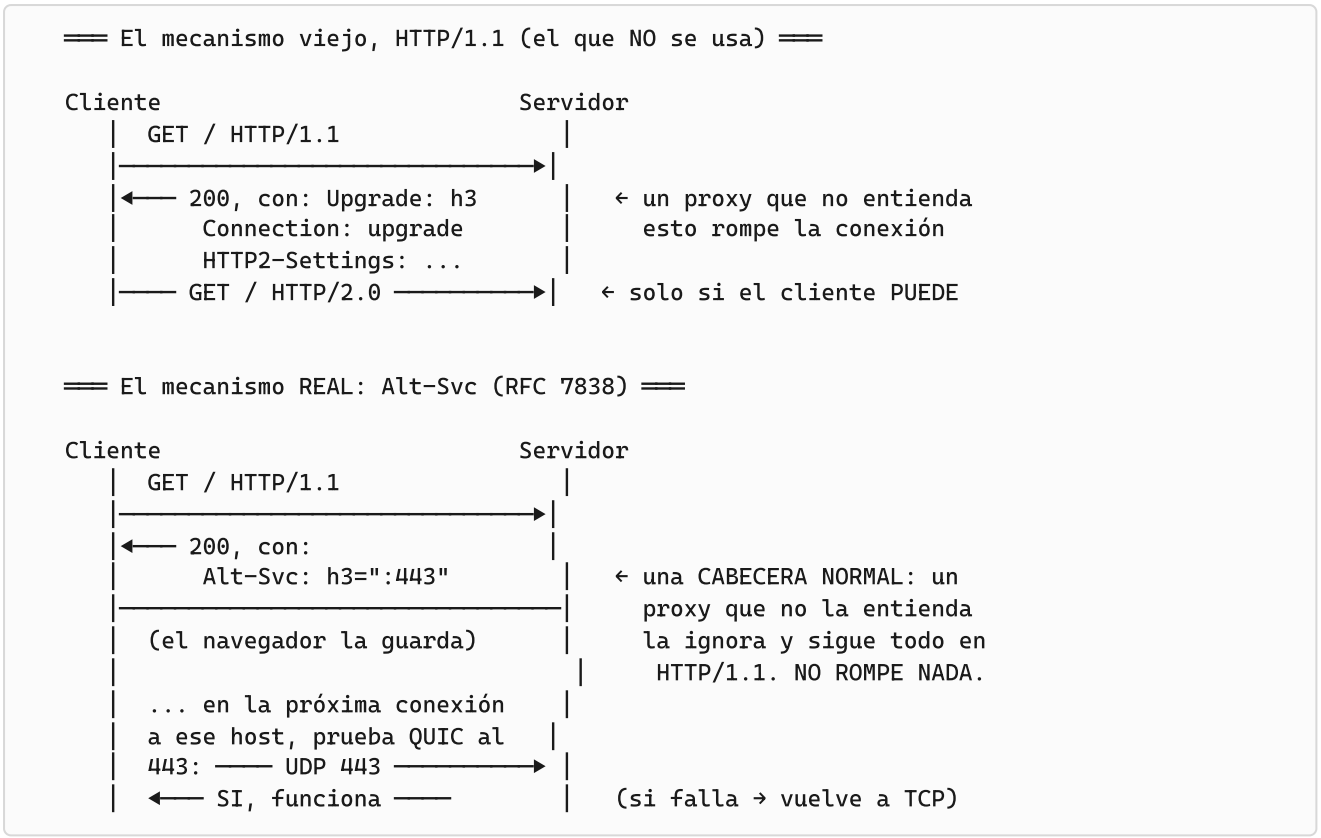
QPACK (`RFC 9204`) es la compresión de cabeceras de HTTP/3. La diferencia con HPACK es conceptual:

```
HPACK   (HTTP/2): el índice se mantiene al ORDENAR las cabeceras.
                  Es SECUENCIAL: para descomprimir el frame N necesito
                  haber descomprimido el N-1.

QPACK   (HTTP/3): los índices se resuelven en CUALQUIER orden.

¿Por qué? Porque en HTTP/3 los frames viajan en streams DISTINTOS, que
llegan al receptor en orden arbitrario. Si la compresión fuera
secuencial, la multiplexación sería imposible.
```

Cómo se negocia HTTP/3: `Alt-Svc` , NO `Upgrade` . Esta es la corrección conceptual más importante frente a 2012:



La diferencia clave: `Upgrade` es un **cambio de protocolo dentro de la conexión actual**, frágil frente a los intermediarios. `Alt-Svc` es **anunciar una alternativa para la próxima conexión**, en una cabecera común que se puede ignorar. El navegador lo guarda y lo prueba después: **cero riesgo, adopción gradual.**

Para descubrir HTTP/3 **antes** de conectar, sin haber ido una vez por HTTP/1.1, están los registros DNS **`SVCB` / `HTTPS` (`RFC 9460`, 2023)**: el DNS ahora transporta **política de transporte** (qué versiones de HTTP soporta, qué IP de *hint*, qué prioridad), no solo direcciones. Es el mismo cambio conceptual que `CAA`. Lo desarrollamos en la Clase 11.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - Modelo **petición/respuesta**, protocolo **sin estado**. - Los **métodos**: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE, CONNECT (+ PATCH). GET/HEAD/OPTIONS/TRACE **seguros**; GET/HEAD **idempotentes**. - Las **5 clases** de códigos (1xx a 5xx) y los del día a día: 200, 201, 204, 206, 301, 302, 304, 307, 308, 400, 401, 403, 404, 405, 408, 429, 500, 502, 503, 504. - `Host` **obligatoria** en HTTP/1.1, y **persistente por defecto**. - `Range` requests y `304 Not Modified`. - **HTTP/1.1 sigue siendo totalmente válido y necesario**: no es «el protocolo viejo», es el *fallback* de todo.

Cambió — y esto es lo que hay que corregir si citás apuntes de 2011: - El curso de 2012 enseñaba `RFC 2616` **(1999)**. La especificación vigente es de **2022**: `RFC 9110` (semántica, *Internet Standard*) + `RFC 9112` (sintaxis de HTTP/1.1). - `RFC 7540` (HTTP/2, 2015) está **OBSOLETA**: la reemplazó `RFC 9113`. - **HTTP/3 (`RFC 9114`)** va sobre **QUIC (`RFC 9000`)**, que **reemplaza a TCP por completo**: no hay handshake de 3 vías, no hay conexión TCP, no hay retransmisión de conexión. La pérdida se resuelve **por stream**. QPACK es `RFC 9204`; QUIC v2 es `RFC 9369`. - La **negociación** es por `Alt-Svc` (`RFC 7838`, 2016), no con

`Upgrade`. - `HPACK` es `RFC 7541` y **sigue vigente**; `QPACK` es `RFC 9204` y es **independiente del orden**, a diferencia de HPACK.

Las cifras reales — y acá está el punto pedagógico del tema:

Versión	Cuota (Cloudflare Radar, *Year in Review 2025*)
HTTP/2	**50 %**
HTTP/1.x	**29 %**
HTTP/3	**21 %**

W3Techs (oct-2026) da ~**40,8 % de los sitios** con soporte de HTTP/3. Navegadores con HTTP/3 por defecto: Chrome/Edge **87** (nov-2020), Firefox **88** (abr-2021), Safari **16** (2020).

> ⚠️ **Trampa metodológica:** si filtrás bots de los datos de Radar, HTTP/3 aparece cerca de **30 %**, porque los bots hablan HTTP/1.x casi siempre. **Siempre declará si incluís bots** si citás una cifra.

Por qué HTTP/3 no se adueñó de la web. Es la pregunta conceptual que vale más que las cifras:

1. **El cambio es gradual y la Web es heterogénea.** HTTP/3 necesita cliente **y** servidor; actualizar el cliente requiere que el usuario cambie de navegador.
2. **Casi todo el tráfico sigue necesitando TCP.** DNS clásico, SSH, servidores de correo, juegos sin migrar. QUIC no reemplaza a TCP: **convive** con él.
3. **Los *middleboxes* son el cuello de botella real.** Cualquier firewall, balanceador o IDS que no entienda QUIC o que filtre UDP/443 **rompe** HTTP/3. Son 25 años de equipos instalados que nadie va a actualizar: HTTP/3 se despliega a la velocidad a la que se reemplazan firewalls.
4. **El costo de adopción de HTTP/2 fue bajo.** Solo cambiaba la capa de aplicación sobre un TCP ya desplegado: bastaba actualizar el servidor web, que es un acto administrativo, no un acto de usuario.

Consecuencia para el examen: si te preguntan «¿cuál es la versión actual de HTTP?», la respuesta completa es **«HTTP/2 es la más usada (50 %); HTTP/3 va en crecimiento (21 %) y no se adueñó de la web, porque el cambio es gradual y casi todo el tráfico sigue necesitando TCP; HTTP/1.1 sigue vivo y es el baseline de comparación»**. Si solo decís «HTTP/3», perdés puntos.

1.10 La tabla que hay que saber de memoria

```
╔═══════════════════════════════════════════════════════════╗
║              QUÉ SIGUE IGUAL EN HTTP/1.1, /2 Y /3         ║
╠═══════════════════════════════════════════════════════════╣
╠═══════════════════════════════════════════════════════════╣
║ 1. Petición / respuesta. Siempre.                         ║
║ 2. Sin estado: el servidor no recuerda peticiones anteriores.    ║
║ 3. Los métodos: GET HEAD POST PUT DELETE OPTIONS TRACE CONNECT. ║
║ 4. Las clases de estado 1xx 2xx 3xx 4xx 5xx.              ║
║ 5. Host obligatoria (en HTTP/2 y HTTP/3 viaja en la       ║
║    pseudo-cabecera :authority).                           ║
║ 6. Persistente por defecto: una conexión, varios recursos.      ║
║ 7. 304 Not Modified y los Range requests.                 ║
║ 8. HTTPS = HTTP sobre TLS. TLS sigue siendo TLS.          ║
╚═══════════════════════════════════════════════════════════╝
```

El punto 5 merece aclaración. En HTTP/2 y HTTP/3 **no hay cabeceras de texto**: los campos viajan en **HPACK** / **QPACK** y se llaman ***pseudo-cabeceras***. `:authority` reemplaza a `Host`, `:method` al método, `:path` a la URL, `:scheme` a `http` / `https`. Todo lo demás (`Content-Type`, `Cookie`, `ETag`, `Cache-Control`) sigue igual, solo que **en minúsculas y comprimido**. Si un examen pide «escribí un mensaje HTTP/2», lo que se espera es la lista de pseudo-cabeceras, no un `GET /... HTTP/1.1` con cabeceras de texto.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Intercambio completo con validación condicional hasta el 304

Enunciado. Un cliente pide `https://www.fing.edu.uy/notas/2026.pdf`. El servidor responde con `ETag`, `Last-Modified` y `Cache-Control: max-age=1800`. Pasados 45 minutos vuelve a pedirlo; el servidor no contesta. Una hora después vuelve a pedirlo y el archivo cambió.

Datos. El recurso pesa **200 000 bytes**. Las cabeceras de respuesta ocupan unos **300 bytes**. El RTT cliente-servidor es de **80 ms**.

Resolución paso a paso.

Paso 1 — Primer pedido: no hay nada en caché.

```
GET /notas/2026.pdf HTTP/1.1
Host: www.fing.edu.uy
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Accept: application/pdf
Accept-Encoding: gzip, deflate
Accept-Language: es-UY,es;q=0.9
Connection: keep-alive
```

Interpretación: `GET` **sin cuerpo** y **sin cabeceras condicionales**: el cliente no tiene nada que ofrecer, así que la petición es completa.

Paso 2 — Respuesta del servidor: `200` *con validadores.*

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 09:00:00 GMT
Server: nginx/1.26.1
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
ETag: "5d9b21-30d40-6518a1f0"
Cache-Control: max-age=1800
Accept-Ranges: bytes
Content-Type: application/pdf
Content-Length: 200000
Connection: keep-alive

%PDF-1.7 ... 200.000 bytes ... %%EOF
```

Interpretación: el cliente guarda **cuatro** cosas: el cuerpo, el `ETag`, el `Last-Modified` y el `Cache-Control`. Anota la **hora de entrada**: 09:00:00.

Paso 3 — ¿Cuándo vence?

```
Vencimiento = hora de entrada + max-age
            = 09:00:00 + 1800 s = 09:00:00 + 30 min = 09:30:00
```

Interpretación: entre las 09:00 y las 09:30 el recurso es **fresco** y el cliente **no vuelve a preguntar**: cero bytes, cero RTT.

Paso 4 — Segundo pedido (09:45): el `max-age` venció, hay que revalidar. El mensaje es **idéntico al del paso 1** más las dos cabeceras condicionales:

```
If-None-Match: "5d9b21-30d40-6518a1f0"
If-Modified-Since: Mon, 05 Oct 2026 09:14:07 GMT
```

Interpretación: `If-None-Match` es la **pregunta condicional**: «si el recurso tiene esta etiqueta, no me lo mandes entero». `If-Modified-Since` es la misma pregunta por fecha. Si el cliente mandó ambos, el servidor **usa el `If-None-Match`**, porque el `ETag` distingue dos versiones del mismo segundo.

Paso 5 — El servidor no contesta.

Interpretación física: el cliente no puede saber si el recurso cambió. El `max-age` venció, así que su copia técnicamente no es válida. Los navegadores reales aplican el **heuristic de frescura**: sin directivas explícitas se usa el 10 % de la edad del recurso. Acá hay `max-age`, así que lo correcto es servir la copia local **marcándola como posiblemente vieja**, que es lo que hace `stale-while-revalidate`.

Paso 6 — Tercer pedido (10:00): ahora sí, y el archivo cambió. La respuesta es un `200` completo **idéntico al del paso 2** salvo tres campos: `Last-Modified` pasa a `Tue, 06 Oct 2026 09:50:11 GMT`, el `ETag` pasa a `"5d9b21-31234-6518a9c8"` y `Content-Length` pasa a `201344`.

Interpretación: como el `ETag` del servidor **no coincide** con el `If-None-Match`, la condición **se ignora** y se manda el `200` completo. El cliente **reemplaza** su copia: nuevo cuerpo, nuevo `ETag`, nueva fecha.

Pedido	Hora	Condición	Respuesta	Bytes de cuerpo	Total
1	09:00	ninguna	`200`	**200 000**	80 ms
2	09:45	`If-None-Match`	*sin respuesta*	0	80 ms
3	10:00	`If-None-Match`	`200` (cambió)	**201 344**	80 ms

```
Datos transferidos:
  pedido 1:  200 000 + 300 ≈ 200 300 B
  pedido 2:        0 + 180 ≈     180 B
  pedido 3:  201 344 + 300 ≈ 201 644 B
  ──────────────────────────────────
  Total:                          402 124 B

RTT totales: 3 × 80 ms = 240 ms
```

Interpretación: el segundo pedido bajó **0 bytes de contenido** en 180 bytes de cabeceras: **99,91 % de ahorro en una petición**. Por eso la caché no es una optimización menor, es la diferencia entre 200 KB y 180 B por revalidación.

⚠️ Errores típicos

- **Olvidar la línea en blanco final** del mensaje de petición: sin ella el servidor no sabe dónde terminan las cabeceras.
- **Poner `Content-Length` en un `304`:** un `304` no lleva cuerpo, y por lo tanto no lleva `Content-Length`.

- **Confundir «el `ETag` no coincide» con «me manda un 404»:** si no coincide, la condición **se ignora** y se manda un `200` completo.
- **Creer que el `304` renueva el TTL del origen:** renueva la **frescura en la caché del cliente**, que vuelve a contar `max-age` desde cero.

2.2 Ejercicio 2 — ¿Qué hace exactamente `Cache-Control: max-age=0` con un ETag presente?

Enunciado. Un servidor responde:

```
HTTP/1.1 200 OK
Date: Tue, 06 Oct 2026 09:00:00 GMT
ETag: "5d9b21-30d40-6518a1f0"
Last-Modified: Mon, 05 Oct 2026 09:14:07 GMT
Cache-Control: max-age=0

<contenido>
```

¿Qué puede y qué no puede hacer el cliente con esto?

Paso 1 — Leer el `max-age=0`. La frescura es de **cero segundos**, o sea **nunca es fresco**: cada vez que el cliente quiera usarlo tiene que revalidar. Pero `max-age=0` **no dice «no lo guardes»** (eso sería `no-store`) ni «no lo devuelvas sin revalidar» (eso sería `no-cache`).

Paso 2 — Lo que SÍ puede hacer el cliente.

```
✅ GUARDAR el recurso en su caché.
✅ Usarlo si el servidor NO responde (fallback, offline).
✅ Aceptar el 304 cuando el servidor confirma que no cambió.
```

Paso 3 — Lo que NO puede hacer.

```
❌ Servirlo de la caché sin preguntarle al servidor.
❌ Asumir que sigue válido porque acaba de bajarlo.
❌ Cachearlo «para siempre» si el servidor nunca más responde.
```

Paso 4 — Y acá está el detalle fino: qué cambia cuando hay `ETag`.

```
SIN ETag y con max-age=0:
  el cliente no puede distinguir "cambió" de "sigue igual"
  → tiene que re-descargarlo ENTERO cada vez.     (200 000 B por uso)

CON ETag y con max-age=0:
  el cliente manda If-None-Match: "5d9b21-30d40-6518a1f0"
  el servidor compara con su ETag actual:
    → si coincide:  304, 0 B de cuerpo
    → si no:        200 completo, 200 000 B

El 304 ocurre, pero NO HAY AHORRO DE RTT: el viaje se sigue entero.
Lo que se ahorra son BYTES.
```

Paso 5 — La cuenta, con números.

```
Recurso de 200 000 B. 5 usos en 1 hora.

Con max-age=3600 (1 hora de frescura):
  uso 1:  200 000 B (descarga)
  uso 2:      0 B (fresco, ni pregunta)
  uso 3:      0 B (fresco, ni pregunta)
  uso 4:      0 B (fresco, ni pregunta)
  uso 5:      0 B (fresco, ni pregunta)
  ──────────────────────────
  Total: 200 000 B        RTT: 1

Con max-age=0 + ETag:
  uso 1:  200 000 B (descarga)
  uso 2:      0 B (304, un RTT)
  uso 3:      0 B (304, un RTT)
  uso 4:      0 B (304, un RTT)
  uso 5:      0 B (304, un RTT)
  ──────────────────────────
  Total: 200 000 B        RTT: 5
```

Interpretación física, y es la frase del examen: **`max-age=0` convierte cada uso en una revalidación, pero gracias al `ETag` cada revalidación cuesta 0 bytes de contenido.** Los bytes totales son los mismos; lo que cambia son los **viajes**: de 1 RTT a 5 RTT. Con un RTT de 200 ms son 800 ms de espera extra.

	`max-age=0` + ETag	`no-cache` + ETag	`no-store`
¿Se guarda en caché?	sí	sí	**no**
¿Hay que revalidar siempre?	sí (0 s de frescura)	**sí, explícitamente**	no aplica
¿Sirve si el servidor cae?	sí	sí	**no**
Diferencia	implícito, por aritmética	explícito, por declaración	—

Respuesta final: `max-age=0` marca el recurso como **vencido de entrada** pero **conservable**: el cliente lo guarda (para poder trabajar sin conexión) y en cada uso revalida contra el servidor. Con `ETag`, esa revalidación es barata en **bytes** (un `304` de 180 bytes) pero **no** economiza el **viaje**. Si lo que se quiere es no revalidar nunca, la directiva correcta es `no-store`.

⚠️ Errores típicos

- **Decir que `max-age=0` equivale a `no-cache`.** En cuanto al comportamiento de uso sí, pero `no-cache` **declara** la intención y `max-age=0` la **logra por aritmética**.
- **Decir que `max-age=0` es «no cachear».** Falso: **sí** se cachea; lo que no existe es la *frescura*.
- **Olvidar el efecto de `Age`.** Si un intermediario sirve con `Age: 30` y `max-age=0`, la frescura es `0 − 30 = −30`: **vencida**, hay que revalidar igual.

2.3 Ejercicio 3 — ¿Qué gana y qué pierde HTTP/3 frente a HTTP/1.1?

Enunciado. Compará HTTP/3 y HTTP/1.1 en cuatro dimensiones: latencia de conexión, comportamiento ante pérdida, aislamiento del tráfico y coste operacional.

Paso 1 — Latencia de conexión, con la cuenta a la vista.

```
HTTP/1.1 sobre TCP + TLS:
  TCP     SYN ──►  ◄── SYN+ACK  ACK ──►        1 RTT
  TLS 1.3 ClientHello ─►  ◄── ServerHello        1 RTT
  ──────────────────────────────────────────────
  Total hasta el primer byte de aplicación:          2 RTT

HTTP/3 sobre QUIC (que ya incluye TLS 1.3):
  QUIC    Initial + Handshake combinado               1 RTT
  ──────────────────────────────────────────────
  Total hasta el primer byte de aplicación:          1 RTT
```

Interpretación: se ahorra **un RTT entero** por conexión nueva. Y como HTTP/3 multiplexa, **no abre una conexión por objeto**: abre una y la reutiliza.

Paso 2 — Pérdida de paquetes.

```
HTTP/1.1:  una conexión = un flujo ordenado.
           si se pierde el segmento 5, el receptor NO puede entregar
           el 6 ni el 7 hasta que llegue el 5 y se retransmite
           (1 RTO con fast retransmit, un RTO entero con timeout).

HTTP/3:    muchos streams independientes, CADA UNO con su numeración
           y su recuperación. Si se pierde el stream 5, el 6 se
           entrega igual. Se reintenta SOLO el 5.
```

Interpretación física: es la diferencia entre **«toda la página quedó congelada 1,5 segundos porque se perdió un byte de una imagen»** y **«esa imagen tarda un poco más; el texto ya se ve»**. El congelamiento total de la página es el síntoma clásico del *head-of-line blocking* de HTTP/1.1.

Paso 3 — Aislamiento del tráfico, el punto donde HTTP/2 queda a medias.

```
HTTP/2:  multiplexa sobre UNA conexión TCP. Cada stream se numera,
         pero TCP es ORDENADO: se pierde un segmento → se frena LA
         CONEXIÓN ENTERA, aunque solo le afecte a un stream.

         ╔═══════════════════════════════════════════╗
         ║ HTTP/2 elimina el HOL blocking A NIVEL DE  ║
         ║ CONEXIÓN, pero NO ENTRE STREAMS.          ║
         ╚═══════════════════════════════════════════╝

HTTP/3:  cada stream tiene su numeración y su recuperación: la
         pérdida de uno no frena a los otros.

         ╔═══════════════════════════════════════════╗
         ║ HTTP/3 elimina el HOL blocking ENTRE STREAMS ║
         ║ porque ya no hay conexión TCP debajo.     ║
         ╚═══════════════════════════════════════════╝
```

Paso 4 — Qué gana y qué pierde HTTP/3.

Gana	Por qué
1 RTT en vez de 2 para conectar	el handshake QUIC incluye el de TLS
Sin *head-of-line blocking* entre streams	recuperación por stream, no por conexión
Migración de conexión	el `Connection ID` sobrevive al cambio de IP y de puerto, lo que arregla el *NAT rebinding* que rompe a TCP
PLPMTUD integrado	mide el camino con su propio tráfico, sin depender de que ICMP no esté filtrado
QPACK sin orden	permite comprimir cabeceras en paralelo, coherente con la multiplexación
0-RTT con reanudación	con tokens anti-replay

Pierde	Por qué
CPU por paquete	cada stream va cifrado con AEAD; el *throughput* en máquinas pequeñas baja
UDP a veces está bloqueado	redes corporativas y operadores filtran UDP/443 o no pasan QUIC
Los *middleboxes* son el cuello de botella	cualquier firewall o IDS que no entienda QUIC **rompe** la conexión
Observabilidad peor	sniffear HTTP/3 exige las claves del endpoint
Menos despliegue	21 % del tráfico de Radar, 40,8 % de los sitios

Paso 5 — El veredicto, que es la frase del examen.

> **HTTP/3 gana en latencia de conexión, en aislamiento de la pérdida y en movilidad; pierde en costo de CPU, en compatibilidad con los equipos que filtran UDP y en despliegue.** La razón por la que no se adueñó de la Web no es técnica —es mejor en casi todo— sino **política de despliegue**: el cambio es gradual, requiere actualizar clientes y servidores, y se choca con 25 años de *middleboxes* instalados que nadie va a cambiar. **HTTP/2 ganó más rápido porque solo cambiaba la capa de aplicación sobre un TCP que ya estaba desplegado.**

⚠️ Errores típicos

- **Decir «HTTP/3 es HTTP/2 sobre UDP». Falso**, y es el error conceptual más caro. HTTP/3 reemplaza TCP **por completo**.
- **Decir que HTTP/2 elimina el HOL blocking sin matizar.** Solo a nivel de conexión; entre streams sigue existiendo.
- **Confundir `Upgrade` con `Alt-Svc`.**
- **Decir «HTTP/3 es la versión actual y la más usada».** La más usada es HTTP/2.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-07, Pregunta 1 (8 puntos) — página HTML con diez imágenes

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1007.pdf **Nota:** la resolución completa con la cuenta de RTT ya está en la **Clase 8 §3.1**. Acá se reproduce desde el ángulo de las **versiones**, que es lo que esta clase agrega.

Enunciado (texto literal del examen)

> **Pregunta 1 (8 puntos)**
>
> Suponga que un usuario intenta cargar en su browser una página HTML. Esta página contiene texto y diez imágenes.
>
> **a)** Describa la secuencia de pasos que ocurren a nivel de capa de aplicación en el cliente y el servidor a los efectos de desplegar la página completa, suponiendo que se utiliza HTTP no persistente.
>
> **b)** ¿Cuál es la diferencia con HTTP persistente? ¿Mejora la performance? Discuta.

Solución oficial (del PDF, textual)

> a)
>
Paso	Responsable	Acción
1	Cliente	Establece la conexión con el servidor
2	Cliente	Envía Request pidiendo la página
3	Servidor	Envía Response con el contenido de la página
4	Servidor	Cierra la conexión
5*	Cliente	Establece la conexión con el servidor
6*	Cliente	Envía Request pidiendo la la imagen 1
7*	Servidor	Envía Response con el contenido de la imagen 1
8*	Servidor	Cierra la conexión
>
> * Se repiten los pasos 5 al 8 para cada una de las imágenes.
>
> b) En el caso de HTTP persistente, se establece la conexión una sola vez, se obtiene el contenido de la página, y a través de la misma conexión se piden todas las imágenes.

> El uso de HTTP persistente puede resultar más performante en los casos en que se utilice una sola conexión por vez (por ejemplo si el servidor nos limita por IP), ya que no existe el overhead de crear nuevas conexiones para cada imagen.
>
> Si el navegador abre conexiones en paralelo para obtener varias imágenes a la vez, probablemente termine de obtener todos los datos más rápidamente, aprovechando mejor el ancho de banda.

a) y b) — Resolución

La tabla oficial es correcta y es la respuesta del apartado a). Lo que hay que agregar son las **capas que la oficial abstrae**: el paso 1 vale **1 RTT de TCP** (2 con HTTPS), el paso 4 es un `FIN` que no cuesta RTT pero deja un `TIME-WAIT` de 2×MSL en el cliente, y el paso 3 debería llevar `Cache-Control` + `ETag` + `Last-Modified`, que es lo que hace que la **segunda** carga de la misma página sea un `304` en lugar de 11 descargas.

Respuesta redactada, lista para copiar en el examen:

> **a)** En HTTP **no persistente**, por cada objeto (el HTML y cada una de las 10 imágenes) el cliente ejecuta **cuatro pasos**: (1) establece una conexión TCP con el servidor, (2) envía un request pidiendo el objeto, (3) el servidor envía un response con el contenido del objeto, (4) el servidor cierra la conexión. El primer objeto es la página HTML; al recibirla y parsearla, el cliente descubre las 10 imágenes referenciadas y repite los pasos 1–4 **una vez por cada imagen**. En total son **11 objetos × 4 pasos = 44 pasos**, con **11 conexiones TCP distintas** y **22 RTT** (2 RTT por objeto: 1 de handshake TCP + 1 de request/response). Si el sitio usa HTTPS, a cada uno de esos RTT se le suma 1 RTT de TLS: **33 RTT**.
>
> **b)** En HTTP **persistente** se establece **una sola** conexión TCP y por ella se piden los 11 objetos uno tras otro: **1 RTT** de handshake + **11 RTT** de request/response = **12 RTT**, contra los 22 del no persistente. La mejora es real cuando el navegador trabaja **secuencialmente**, porque se eliminan los `N − 1 = 10` handshakes. **Pero no siempre mejora:** si el navegador abre conexiones **en paralelo** (que es lo que hacen los reales) y los objetos vienen de **varios servidores**, los handshakes pasan todos al mismo tiempo: el no persistente baja a **4 RTT** y el persistente queda en **4 RTT**, o sea, la ventaja **desaparece**.

Lo que esta clase 9 agrega: la misma pregunta, versión por versión.

```
=== El mismo caso, en HTTP/2 ===
1 conexión TCP (11 streams simultáneos)        1 RTT
GET HTML + 10 GETs en paralelo                 1 RTT
──────────────────────────────────────────────────
Total:                                           2 RTT
Contra los 22 RTT del no persistente y los 12 del persistente.
La pérdida de un stream frena SOLO a ese stream.

=== El mismo caso, en HTTP/3 ===
1 conexión QUIC (con TLS integrado)            1 RTT
GET HTML + 10 GETs en paralelo                 1 RTT
──────────────────────────────────────────────────
Total:                                           2 RTT
Pero el handshake es 1 RTT en vez de 2 (TCP + TLS), y la pérdida de un
stream no frena a los otros: en HTTP/1.1 una sola pérdida de paquete en
una imagen congelaba la página entera durante 1 RTO.

=== Y con HTTPS, que es el caso real ===
HTTP/1.1 no persistente + HTTPS:  11 × (1 TCP + 1 TLS + 1 HTTP) = 33 RTT
HTTP/1.1 persistente  + HTTPS:     1 + 1 + 11                    = 13 RTT
HTTP/2              + TLS:         1 + 1                         =  2 RTT
HTTP/3              + QUIC:        1                             =  1 RTT
```

Interpretación: el objetivo de HTTP/2 y HTTP/3 **es exactamente este problema de examen**. Los dos nacen para matar el número de viajes que el enunciado cuenta, y la lección es que **el problema no se resolvió cambiando la forma de pedir**: se resolvió cambiando la forma de **transportar** varias peticiones a la vez.

> **Nota contra la oficial:** la oficial dice «puede resultar más performante», que es la respuesta honesta y matizada, y agrega el mejor argumento del apartado b): **«si el servidor nos limita por IP»**, o sea que el navegador no puede paralelizar porque el servidor impone un límite por IP. Yo repito ese argumento y agrego las tres precisiones que la oficial omite: el costo en RTT del paso 1, el `TIME-WAIT` del paso 4 y el `ETag` / `Cache-Control` del paso 3.

3.2 Examen 2009-07, Problema 2 — HTTP como protocolo de consulta

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009, Problema 2, apartado b). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf **Nota:** la resolución completa del problema está en la **Clase 8 §3.2**. Acá se desarrolla el ángulo de **cómo se escribe un mensaje HTTP/1.1 válido**.

Enunciado (apartado b)

> Implementar `bool isSpam(string ipAddress, string subject)` que devuelve `true` si **naivespamming** considera que el correo es spam.

Solución oficial (del PDF)

```
Socket s = new Socket("www.naivespamming.org", 80);
if (!s.connect()) return false;
s.out.write(
  "GET isspam.cgi?sender=" + ipAddress +
  "&subject=" + queryStringStuff(subject) + " HTTP/1.1\r\n\r\n");
string response = s.in.readLine();
s.close();
string[] result = response.split();
return result[1] == "200";
```

Lo que hay que corregir, mensaje HTTP adentro

El mensaje que la oficial escribe **no es un mensaje HTTP/1.1 válido**:

❌ Lo que escribe la oficial

```
GET isspam.cgi?... HTTP/1.1
\r\n\r\n
```

✅ Lo que debería escribir

```
GET /isspam.cgi?... HTTP/1.1
 Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close
\r\n\r\n
```

Defecto	Por qué importa
`Host` ausente	Es **obligatoria** en HTTP/1.1. Sin ella, un servidor con *virtual hosting* no sabe a qué sitio le estás pidiendo, y muchos responden `400` o sirven el sitio por defecto.
Path relativo (`isspam.cgi`)	El `request-target` de HTTP/1.1 tiene que ser un **absolute path** que empiece por `/`.
`\r\n\r\n` sí está bien	Ese es el `CRLF` final que cierra las cabeceras. Es lo único que la oficial acertó.
Sin `Accept`	El servidor no sabe qué formato querés y puede devolver HTML en vez de texto.
Sin `Connection: close`	Como no pidió `Content-Length` ni pidió cierre, el cliente queda esperando bytes que no llegan.

GET contra POST: la discusión que el enunciado no pide pero suma

La oficial usa `GET`. ¿Corresponde? Buscar en una base de datos es una **lectura** (`GET` está bien); registrar que un remitente hizo spam es una **escritura** (`POST`). El enunciado dice «considera que el correo es spam», o sea que es una consulta, así que `GET` es defendible. **PERO** el parámetro `subject` puede llevar información sensible: mandarlo en la URL lo deja en el historial del navegador, en los logs del servidor y en los proxies. Con `POST` iría en el cuerpo.

```
=== Con GET ===
GET /isspam.cgi?sender=203.0.113.7&subject=Venta%20de%20autos HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Accept: text/plain
Connection: close

=== Con POST ===
POST /isspam.cgi HTTP/1.1
Host: www.naivespamming.org
User-Agent: FING-SpamFilter/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 46
Accept: text/plain
Connection: close

sender=203.0.113.7&subject=Venta%20de%20autos
```

Y el parseo, que la oficial hace mal: `response.split()` toma el primer espacio y asume que `result[1]` es el código, lo que se rompe con una redirección. Lo robusto es **parsear la primera línea** (`HTTP/<ver> <código> <frase>`), seguir la redirección si la hay, y recién ahí interpretar el `2xx` o el `4xx`.

La corrección semántica del `return`

El enunciado dice que **404 = es spam** y **200 = es legítimo**. La oficial devuelve `true` con el `200`: eso **invierte la semántica** y marca como spam un correo que el servidor dijo que es limpio.

```
código == 200  →  es legítimo  →  return false
código == 404  →  es spam      →  return true
otro / error   →  no se sabe   →  return false   (fail-open)
```

> **Nota explícita:** la solución oficial contradice al enunciado. En un examen, **respetá el enunciado** y **señalá la contradicción por escrito**. Eso suma puntos: demuestra que leíste los dos documentos.

Lo que este problema aporta a la clase: un `4xx` no es un error técnico, es **información de negocio** para la aplicación que consume HTTP. **HTTP no solo transporta contenido: transporta semántica.**

3.3 Examen 2008-12, Pregunta 4 (7 puntos) — el mismo patrón de diseño

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Este problema se resuelve **entero en la Clase 13**, junto con el correo. Acá lo dejo anotado porque es el **mismo patrón de diseño** que usa `Transfer-Encoding: chunked`, y entenderlo dos veces lo hace familia:

```
POP3:   el fin de mensaje es  CRLF . CRLF
        si un punto aparece al principio de una línea del mensaje, se
        DUPLICA. Al recibir, se quita el punto extra.

HTTP:   el fin de un fragmento chunked es  0<CRLF>
        si un dato empieza con "0" seguido de "<CRLF>", hay que
        escaparlo, porque si no el cliente lo toma como fin de cuerpo.
```

La idea común, que es la que hay que saber decir: cuando **el delimitador vive dentro del mismo canal que los datos** (no hay longitud explícita), hace falta **escapar** el delimitador en el emisor y **desescapar** en el receptor. Es el mismo problema que el **byte stuffing** del PPP (`2003-03 P3`) y el **bit stuffing** de Ethernet. **Tres protocolos, tres objetos de relleno distintos: el punto en POP3 y SMTP, el byte de escape en PPP, el bit en Ethernet.** La razón es siempre la misma: **ninguno de esos caracteres puede aparecer legítimamente donde se lo proscribe.**

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. Un **método** declara la intención. **Seguro** = no cambia el estado, **idempotente** = repetirlo *n* veces es lo mismo que una. La distinción decide si el cliente puede **reintentar solo**.
2. `GET` / `HEAD` / `OPTIONS` / `TRACE` son seguros; `GET` / `HEAD` / `PUT` / `DELETE` son idempotentes; `POST` **no es ninguna de las dos**.
3. En `GET` los datos van **en la URL** (por eso quedan en el historial y los logs); en `POST` van **en el cuerpo** (por eso no).
4. `Host` es **obligatoria** en HTTP/1.1 y es lo que hace posible el *virtual hosting* y, por lo tanto, lo que impide esconder HTTP detrás de un NAT.
5. `Cache-Control`: `no-store` (no guardar), `no-cache` (guardar pero revalidar), `max-age=N` (fresco N segundos), `private` / `public` (quién puede cachear), y `Vary` (de qué depende la variación).
6. La validación condicional es `ETag` + `If-None-Match` (o `Last-Modified` + `If-Modified-Since`); si coincide, el servidor responde `304` **sin cuerpo y sin** `Content-Length` y **renueva la frescura** de la caché del cliente.
7. **Proxy** reenvía, **gateway** traduce protocolos, **CDN** copia el contenido cerca del usuario: el CDN mejora la latencia **reduciendo la distancia al usuario**, y como la carga de una página es `viajes × RTT`, eso la acelera.
8. **HTTP/2** (`RFC 9113`) multiplexa streams sobre **una conexión TCP** y elimina el HOL blocking **de conexión** (no entre streams); **HTTP/3** (`RFC 9114` sobre **QUIC**, `RFC 9000`) **reemplaza TCP por completo** y elimina el HOL blocking **entre streams**. Se negocia con `Alt-Svc`, no con `Upgrade`.
9. Las cifras reales: **HTTP/2 50 %, HTTP/1.x 29 %, HTTP/3 21 %** (Cloudflare Radar 2025). **HTTP/3 no se adueñó de la web**, y la razón es de despliegue, no técnica.
10. Lo que **no cambia nunca**: petición/respuesta, sin estado, los métodos, las clases 1xx–5xx, `Host` obligatoria (que pasa a `:authority` en HTTP/2 y 3), persistente por defecto, `304` y `Range`.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un cliente hace `POST /pagar` y no recibe respuesta. ¿Puede reintentar automáticamente? ¿Por qué sí o por qué no?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Qué diferencia hay entre `Cache-Control: no-cache`, `max-age=0` y `no-store`?

<details>
<summary>Respuesta</summary>
</details>

3. Escribí la respuesta a una petición que llega con `If-None-Match: "5d9b21-30d40-6518a1f0"` y el servidor tiene ese mismo `ETag`. ¿Qué cabeceras **no** puede llevar?

▶ Respuesta

4. Un `GET` con `Cache-Control: max-age=0` y un `ETag` presente. ¿Cuánto se ahorra en cada revalidación y cuánto **no** se ahorra?

▶ Respuesta

5. ¿Por qué un CDN mejora la latencia? Explicá sin usar la palabra «caché».

▶ Respuesta

6. ¿Qué gana y qué pierde HTTP/3 frente a HTTP/1.1? Nombrá al menos dos de cada.

▶ Respuesta

7. ¿HTTP/2 elimina el head-of-line blocking? La respuesta precisa, porque «sí» y «no» están las dos.

▶ Respuesta

8. ¿Por qué HTTP/3 no se adueñó de la web si es técnicamente mejor? ¿Es un problema técnico?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1007` / `sirc1007`** — Pregunta 1 (8 pts): página HTML con texto y 10 imágenes, HTTP no persistente vs. persistente
- **Examen `eirc0907` / `sirc0907`** — Problema 2, apartado b): `isSpam()` sobre HTTP, construcción del mensaje y parseo del código
- **Examen `eirc0812` / `sirc0812`** — Pregunta 4 (7 pts): el `CRLF.CRLF` de POP3 (desarrollada en la Clase 13)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.3 («El protocolo de la Web: HTTP»), §7.3.5 (caché) y §7.3.6 (CDN, *content delivery networks*)
- **RFC 9110** — HTTP Semantics (2022, *Internet Standard*; sustituye a RFC 2616) · **RFC 9112** — HTTP/1.1 (2022, sintaxis)
- **RFC 9113** — HTTP/2 (2022; obsoleta RFC 7540) · **RFC 9114** — HTTP/3 (2022)
- **RFC 7541** — HPACK, compresión de cabeceras de HTTP/2 (vigente) · **RFC 9204** — QPACK, compresión de HTTP/3 (independiente del orden)
- **RFC 9218** — *Extensible Prioritization Scheme* (EPS): reemplaza la prioridad deprecada de RFC 7540
- **RFC 9000** / **9001** / **9002** — QUIC: transporte, TLS sobre QUIC, recuperación y control de congestión (mayo-2021) · **RFC 9369** — QUIC v2
- **RFC 7838** — *Alternative Services* (`Alt-Svc`): cómo se anuncia HTTP/3 (2016)

- **RFC 6265** — HTTP State Management Mechanism (cookies) · **RFC 9460** — SVCB/HTTPS: política de transporte en el DNS
- **Cloudflare Radar**, *Year in Review 2025* — HTTP/2 50 %, HTTP/1.x 29 %, HTTP/3 21 % · **W3Techs** (oct-2026) — ~40,8 % de los sitios con HTTP/3
- **Clase siguiente:** 10 — DNS: la base de datos distribuida